

RoK4 Flat RSL-RL Task 구조 문서

작성일: 2026-07-15
최종 업데이트: 2026-08-24
대상 저장소: RoK4 repository root (\${ROK4_LAB_ROOT})
기준 환경: Isaac Lab v2.3.2, Isaac Sim 5.1.0, env_isaaclab

경로 표기

문서의 저장소 경로는 개인 PC의 절대 경로 대신 다음 변수를 사용한다. 다른 위치에 clone했다면 오른쪽 경로만 실제 위치에 맞게 바꾼다.

```
export ROK4_LAB_ROOT="${HOME}/rok4_lab"
export ISAACLAB_ROOT="${HOME}/IsaacLab"
```

개요

이번 작업의 목적은 RoK4를 위한 첫 강화학습 환경을 만드는 것이다. 바로 rough terrain으로 가지 않고, 먼저 flat terrain에서 blind velocity walking을 학습할 수 있는 최소 구조를 만들었다.

중요한 점은 G1을 그대로 복사한 것이 아니라는 점이다. G1은 잘 정리된 Isaac Lab humanoid locomotion task 구조를 참고한 템플릿이고, 실제 로봇 asset, 관절 순서, action dimension, reward body 이름은 모두 RoK4 기준으로 다시 작성했다.

ADAPT 행렬, actions.py 와 actuator의 객체 관계, compute() 입력 target/state 출처, velocity target과 None 처리, torque limit을 포함한 상제 제어 흐름은 docs/rok4_adapt_control_structure_ko.rst 와 생성된 docs/_build/pdf/rok4_adapt_control_structure_ko.pdf 에 별도로 정리한다. 이 문서는 전체 task 구조와 연결 관계를 중심으로 설명한다.

Observation noise, reset state randomization, physics DR의 정확한 범위와 표본화 시점, Train/Play/Teleop 차이는 docs/rok4_randomization_and_noise_ko.rst 와 생성된 docs/_build/pdf/rok4_randomization_and_noise_ko.pdf 를 기준 문서로 사용한다.

Concurrent body-frame base-velocity estimator의 225D 입력, 3D target, PPO gradient 분리, RMSE logging, checkpoint와 fused 240D ONNX export는 docs/rok4_concurrent_state_estimator_ko.rst 와 생성된 docs/_build/pdf/rok4_concurrent_state_estimator_ko.pdf 에 정리한다.

현재 actuator-space 기준 정책은 2026-07-24_19-34-26_symmetry_aug_nojumps2_swing_roll100_fresh/model_9999.pt 이며, 이 문서에서는 이를 experimental Yunho Symmetry ADAPT v1 baseline으로 기록한다. 이 checkpoint를 보존한 상태에서 다음 단계로 domain randomization과 policy observation noise를 조정한다. 이전 2026-07-15_17-28-41/model_4999.pt 는 joint-space Yunho v1 reference이므로 ADAPT action interface와 checkpoint 호환성이 없다.

현재 생성된 task는 다음 세 개다.

Task 이름	목적
RoK4-Isaac-Velocity-Flat-v0	rok4_train.usd 를 사용하는 RoK4 flat-ground velocity tracking 학습용 task
RoK4-Isaac-Velocity-Flat-Play-v0	visual mesh가 있는 rok4_test.usd 로 학습된 RoK4 flat policy를 재생하는 task
RoK4-Isaac-Velocity-Flat-Teleop-v0	rok4_test.usd 와 Isaac Lab SE(2) gamepad/keyboard 입력으로 학습된 policy를 조종하는 task

전체 구조

현재 RoK4 Lab 저장소의 핵심 구조는 다음과 같다.

```

rok4_lab/
  assets/
    rok4_wholebody/          # URDF/USD/STL asset bundle, git ignore
  source/
    rok4_tasks/
      rok4_tasks/
        __init__.py          # RoK4 task registration import
      assets/
        robots/
          rok4.py             # RoK4 asset, actuator, joint order 정의
          rok4_adapt.py       # ADAPT 행렬과 actuator-space explicit PD
        manager_based/
          locomotion/
            velocity/
              mdp/
                actions.py     # actuator action -> mapped joint target
                commands.py    # episode role과 비동기 exact-zero 정지 command
                events.py       # correlated foot friction과 joint reset DR
                observations.py  # joint state -> actuator state
                rewards.py      # RoK4 actuator-space reward 계산
                symmetry.py     # 좌우 observation/action data augmentation
            config/
              rok4/
                __init__.py     # Gym task 등록
                flat_env_cfg.py # RoK4 Flat env/reward/action/obs 정의
                contact_force_visualizer.py # 발 접촉력 화살표/숫자/그래프 debug view
                push_test_window.py # Play/Teleop command/실제 속도 숫자 표시와 수동 push UI
                domain_randomization_cfg.py # RoK4 DR 범위와 event mode 정의
                agents/
                  rsl_rl_ppo_cfg.py # RSL-RL PPO 설정
      scripts/
        check_rok4_zero.py     # 모델/PD zero hold 검사
        check_rok4_random.py   # sinusoidal command 검사
        check_rok4_joint_monkey.py # joint axis/limit 검사
        rsl_rl/
          _run_isaaclab_rsl.py  # Isaac Lab 원본 train/play wrapper
          rok4_ppo.py           # KL + velocity estimator PPO/runner/exporter
          train.py              # RoK4 task/runner 등록 후 Isaac Lab train 실행
          play.py               # RoK4 task 등록 후 Isaac Lab play 실행
          play_teleop.py        # gamepad/keyboard command를 주입하는 teleop 실행

```

RoK4 구조 관계

현재 RoK4 flat task는 flat_env_cfg.py 를 중심으로 움직인다. 다만 실행 entry point는 config/rok4/__init__.py 의 Gym task 등록이고, flat_env_cfg.py 는 환경 설정의 main config 역할을 한다.

```

[사용자 실행]
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py
|
v
scripts/rsl_rl/train.py
|
v
scripts/rsl_rl/_run_isaacclab_rsl.py
|   RoK4 package path 추가
|   Isaac Lab 원본 rsl_rl/train.py 실행 전 import rok4_tasks 삽입
|   train에서는 RoK4OnPolicyRunner 삽입
v
source/rok4_tasks/rok4_tasks/__init__.py
|
v
manager_based/locomotion/velocity/config/rok4/__init__.py
|   Gym task 등록
|   id = RoK4-Isaac-Velocity-Flat-v0
|
+--> env_cfg_entry_point
|   flat_env_cfg.py : RoK4FlatEnvCfg
|
+--> rsl_rl_cfg_entry_point
|   agents/rsl_rl_ppo_cfg.py : RoK4FlatPPORunnerCfg

```

환경 설정 관계는 다음과 같다.

전체 class 상속/사용 관계

아래 트리에서 상속 은 Python class inheritance이고, 포함/사용 은 config 객체를 만들거나 참조하는 관계다. 따라서 `rok4.py` 는 `rok4_adapt.py` 를 상속하지 않고, 그 파일의 config class를 import해 `ROK4_TRAIN_CFG` 안에 포함한다.

```

Isaac Lab
├── ArticulationCfg
│   ├── instance: ROK4_TRAIN_CFG [rok4.py]
│   │   ├── spawn/init_state/joint physical properties
│   │   └── actuators["body"]
│   │       ├── instance: RoK4AdaptActuatorCfg(...) [rok4_adapt.py의 class 사용]
│   │       │   ├── link_alpha/link_beta
│   │       │   ├── actuator Kp/Kd
│   │       │   ├── actuator torque/velocity limits
│   │       └── torque/velocity limit factors
├── IdealPDActuatorCfg
│   ├── DelayedPDActuatorCfg
│   │   ├── 자식(상속): RoK4AdaptActuatorCfg [rok4_adapt.py]
│   │   └── class_type = RoK4AdaptActuator
├── IdealPDActuator
│   ├── DelayedPDActuator
│   │   ├── 자식(상속): RoK4AdaptActuator [rok4_adapt.py]
│   │   │   ├── 포함: RoK4AdaptTransmission
│   │   │   │   ├──  $q = J \psi$ 
│   │   │   │   ├──  $\psi = J^{-1} q$ 
│   │   │   │   └──  $\tau_q = J^{-T} \tau_\psi$ 
│   │   └── override: compute()
│   │       ├── joint state -> actuator state
│   │       ├── actuator-space explicit PD
│   │       ├── actuator torque limit clip
│   │       └── actuator torque -> PhysX joint torque
├── ActionTermCfg
│   ├── 자식(상속): RoK4ActuatorPositionActionCfg [mdp/actions.py]
├── ActionTerm
│   ├── 자식(상속): RoK4ActuatorPositionAction [mdp/actions.py]
│   └── raw action ->  $\psi_{target}$  ->  $q_{target}$ 
├── ObservationGroupCfg
│   ├── 자식(상속): RoK40bservationsCfg.PolicyCfg [flat_env_cfg.py]
├── LocomotionVelocityRoughEnvCfg [Isaac Lab velocity_env_cfg.py]
│   ├── 자식(상속): RoK4FlatEnvCfg [flat_env_cfg.py]
│   │   ├── 포함: RoK4ActionsCfg
│   │   ├── 포함: RoK40bservationsCfg
│   │   ├── 포함: RoK4RewardsCfg
│   │   ├── 포함: RoK4TerminationsCfg
│   │   ├── scene.robot = ROK4_TRAIN_CFG
│   │   ├── 자식(상속): RoK4FlatEnvCfg_PLAY
│   │   │   └── 자식(상속): RoK4FlatEnvCfg_TELEOP
│   │   └── Gym task 등록 -> ManagerBasedRLEnv
├── RewardsCfg [Isaac Lab velocity_env_cfg.py]
│   ├── 자식(상속): RoK4RewardsCfg [flat_env_cfg.py]
├── TerminationsCfg [Isaac Lab velocity_env_cfg.py]
│   ├── 자식(상속): RoK4TerminationsCfg [flat_env_cfg.py]

RSL-RL
├── ActorCritic
│   ├── 자식(상속): RoK4EstimatorActorCritic [scripts/rsl_rl/rok4_ppo.py]
│   └── 포함: 225D -> 3D base-velocity estimator
├── PP0
│   ├── 자식(상속): RoK4PP0 [scripts/rsl_rl/rok4_ppo.py]
├── OnPolicyRunner
│   ├── 자식(상속): RoK40nPolicyRunner [scripts/rsl_rl/rok4_ppo.py]
│   └── RoK4PP0 생성 + KL/estimator logging/checkpoint

RoK4 local MDP
├── actions.py
│   ├── RoK4ActuatorPositionActionCfg
│   └── RoK4ActuatorPositionAction
│       ├── clipped raw actuator action 보관
│       ├──  $\psi_{default} + \text{scaled action} \rightarrow \psi_{target}$ 
│       └──  $J \psi_{target} \rightarrow q_{target}$ 
├── observations.py
│   ├── actuator_pos_rel:  $J^{-1} (q - q_{default})$ 
│   ├── actuator_vel_rel:  $J^{-1} (\dot{q} - \dot{q}_{default})$ 
│   └── base/Foot height: flat env origin 기준 world Z

```

```

├── Foot contact flag/current air time
├── rewards.py
│   ├── actuator torque/velocity/acceleration penalty
│   ├── actuator torque/velocity limit penalty
│   ├── mapped joint action-position limit penalty
│   ├── 접촉 중 foot flat-orientation penalty (구현됨, 현재 None)
│   ├── yaw-frame straight-walking stance-width penalty (구현됨, 현재 None)
│   ├── base-height L2와 swing-foot clearance reward
│   ├── signed foot lateral-separation anti-cross penalty
│   └── 1차/2차 clipped raw actuator action-rate penalty
├── symmetry.py
│   ├── 240D term-major policy history 좌우 반전
│   ├── 240D clean critic history 좌우 반전
│   ├── 10D current privileged base/Foot state 좌우 반전
│   └── 13D raw ADAPT actuator action 좌우 반전
├── RoK4 debug/verification
│   ├── ContactSensor
│   │   ├── 자식(상속): RoK4ContactForceVisualizer [contact_force_visualizer.py]
│   │   ├── env-0 좌우 발 world-frame GRF 화살표
│   │   ├── 좌우 발 force magnitude 숫자 panel
│   │   └── 좌우 ``|F|`` 최근 300 UI sample live plot과 simulation-time X축
│   ├── ManagerBasedRLEnvWindow
│   │   ├── 자식(상속): RoK4PushTestWindow [push_test_window.py]
│   │   ├── 선택 env의 command/실제 vx/vy/vz/wz/|vxy| 숫자 표시
│   │   ├── Play/Teleop의 base-yaw-frame root delta-v 버튼
│   │   ├── Window > IsaacLab panel 복구와 Property tab 재도킹
│   │   └── 다음 policy step에서 선택 env에 push 적용
│   ├── domain_randomization_cfg.py
│   │   └── material/mass/COM/external wrench/reset DR 설정
│   ├── check_rok4_zero.py
│   │   └── passive/zero-command/torque-hold 검사
│   ├── check_rok4_random.py
│   │   └── 작은 sinusoidal target으로 전체 actuator/PD 검사
│   └── check_rok4_joint_monkey.py
│       ├── joint axis/order/position-limit 검사
│       └── teleport 또는 ADAPT actuator torque-PD sweep

```

runtime actuator 생성 관계는 다음과 같다. 이 구간은 class 상속이 아니라 config 객체 전달이다.

```

rok4.py
ROK4_ADAPT_LINK_ALPHA/BETA, gain, limit 정의
-> RoK4AdaptActuatorCfg(...) 생성
-> ROK4_TRAIN_CFG.actuators["body"]에 저장
-> Isaac Lab Articulation이 actuator_cfg.class_type 호출
-> RoK4AdaptActuator(cfg=actuator_cfg) 생성
-> RoK4AdaptTransmission(cfg.link_alpha, cfg.link_beta) 생성
-> 모든 environment에 ADAPT actuator-space PD 적용

```

상속, config 기본값, override의 차이

이 구조에는 서로 다른 네 가지 관계가 함께 있으므로 구분해서 읽어야 한다.

관계	예시	의미
class 상속	RoK4AdaptActuator(DelayedPDActuator)	부모의 delay buffer/reset과 actuator interface를 물려받고 ADAPT-space compute()를 재정의
config class 상속	RoK4AdaptActuatorCfg(DelayedPDActuatorCfg)	부모 config의 gain과 min_delay/max_delay field에 ADAPT field를 추가
config 객체 생성/override	RoK4AdaptActuatorCfg(link_alpha=ROK4_ADAPT_LINK_ALPHA, ...)	rok4.py가 config 기본값 중 RoK4에서 실제 사용할 값을 명시적으로 덮어씀
runtime 포함	ROK4_TRAIN_CFG.actuators["body"]	완성된 actuator config 객체를 robot asset config가 보관하고 Isaac Lab Articulation이 runtime actuator를 생성

rok4.py 와 rok4_adapt.py 사이에는 상속 관계가 없다. rok4.py 가 RoK4AdaptActuatorCfg 를 import하고 config 객체를 만들어 사용하는 관계다.

ADAPT 링크 값 전달 예시

ROK4_ADAPT_LINK_ALPHA 가 cfg.link_alpha 로 바뀌는 별도 대입문은 보이지 않는다. @configclass 가 생성한 constructor의 keyword argument 대입으로 같은 동작이 이루어진다.

```
rok4.py
ROK4_ADAPT_LINK_ALPHA = 0.09845
-> RoK4AdaptActuatorCfg(link_alpha=ROK4_ADAPT_LINK_ALPHA)
-> actuator_cfg.link_alpha = 0.09845
-> Isaac Lab Articulation:
    actuator_cfg.class_type(cfg=actuator_cfg, ...)
-> RoK4AdaptActuator.__init__(cfg=actuator_cfg)
-> RoK4AdaptTransmission(link_alpha=cfg.link_alpha)
-> self.link_alpha = 0.09845
-> ratio = link_beta / link_alpha
-> J, J-1, JT, J-T 생성
```

RoK4AdaptActuatorCfg 의 link_alpha=0.09845, link_beta=0.06, torque_limit_factor=0.9, velocity_limit_factor=0.9 은 caller가 값을 생각했을 때 사용하는 fallback 기본값이다. 현재 ROK4_TRAIN_CFG 는 이 값들을 모두 명시적으로 전달하므로 rok4.py 의 값이 우선한다. 반면 expected_joint_names, actuator_torque_limit, actuator_velocity_limit 은 MISSING 이므로 반드시 외부 config에서 넣어야 하며, 빠뜨리면 초기화 단계에서 잘못된 설정으로 처리된다.

Gain 값 전달과 실제 사용

Gain은 Gym YAML처럼 canonical actuator 순서를 쉽게 확인할 수 있도록 먼저 13개 list로 정의하고, Isaac Lab config가 joint name으로 안전하게 resolve하도록 dictionary로 변환한다. USD 내부 순서는 좌우 joint가 교차되어 있으므로 이름 dictionary와 runtime canonical reorder가 단순 list 직접 전달보다 안전하다.

```
ROK4_ACTUATOR_KP_VALUES / ROK4_ACTUATOR_KD_VALUES
-> _make_joint_dict(...)
-> ROK4_ACTUATOR_KP / ROK4_ACTUATOR_KD
-> RoK4AdaptActuatorCfg(stiffness=..., damping=...)
-> DelayedPDActuator 초기화: gain/effort state + command DelayBuffer
-> RoK4AdaptActuator.compute()에서 canonical actuator 순서로 변환
-> tau_psi = Kp (psi_target - psi) - Kd psi_dot
```

현재 actuator-interface 설정은 hip yaw/roll의 좌우 stiffness는 대부분 유지하면서 ADAPT-coupled hip pitch/knee 계열을 더 낮춰 착지 compliance를 시험한다. 이 값은 joint-space gain이 아니며 RoK4AdaptActuator.compute() 의 psi 오차와 속도에 적용된다. 가운데 pair는 ADAPT 행렬을 통해 ankle pitch에도 기여한다.

```
한쪽 다리 Kp: [240, 240, 120, 120, 40, 40]
한쪽 다리 Kd: [12, 12, 12, 12, 4, 4]
Torso yaw:    Kp=100, Kd=5
```

ROK4_KP 와 ROK4_KD 는 이전 이름을 import하는 코드의 호환 alias다. 현재 ROK4_TRAIN_CFG 의 실제 제어 경로는 ROK4_ACTUATOR_KP 와 ROK4_ACTUATOR_KD 를 직접 사용한다.

Actuator limit 값 전달과 적용

최대 limit list는 config를 통해 runtime actuator에 전달된 뒤 environment 수만큼 GPU tensor로 확장된다.

```
actuator maximum torque list
-> cfg.actuator_torque_limit
-> actuator_torque_limit_max tensor [num_envs, 13]
-> actuator_torque_limit = maximum * torque_limit_factor

actuator maximum velocity list
-> cfg.actuator_velocity_limit
-> actuator_velocity_limit_max tensor [num_envs, 13]
-> actuator_velocity_limit = maximum * velocity_limit_factor
```

현재 두 factor는 모두 0.9 이지만 서로 독립적으로 조정할 수 있다. actuator_torque_limit 은 PD가 요청한 tau_psi 를 실제로 clip하며 torque-limit reward의 기준도 된다. actuator_velocity_limit 은 Gym 코드와 같이 velocity-limit reward의 초과량 검사 기준이며 velocity state 자체를 강제로 잘라내지는 않는다. Runtime에는 두 maximum tensor와 두 factor가 적용

된 limit tensor만 저장한다. Actuator torque를 joint torque로 변환한 뒤에는 ROK4_JOINT_TORQUE_LIMITS_SIM 이 PhysX solver의 최종 안전 상한으로 적용된다. 이 값은 joint mechanical maximum을 그대로 사용하며 별도의 0.9 factor를 적용하지 않는다. 별도의 actuator-position soft limit은 추가하지 않았고, ADAPT mapping 뒤 joint target과 실제 joint position에는 기존 joint 95% soft position limit 검사가 유지된다.

환경 config의 포함 관계

```

RoK4FlatEnvCfg
├─ 상속: IsaacLab LocomotionVelocityRoughEnvCfg
├─ 포함: actions = RoK4ActionsCfg()
├─ 포함: observations = RoK40bservationsCfg()
├─ 포함: rewards = RoK4RewardsCfg()
├─ 포함: terminations = RoK4TerminationsCfg()
├─ 학습 사용: ROK4_TRAIN_CFG
├─ 사용: ROK4_JOINT_ORDER
├─ 사용: ROK4_ACTUATOR_ACTION_SCALE
├─ 호출: apply_rok4_domain_randomization(self)

RoK4ActuatorPositionAction
├─ clipped raw actuator action 유지
├─ psi_target = psi_default + scale * action
├─ q_target = J * psi_target

RoK4AdaptActuator
├─ q, qdot -> psi, psi_dot
├─ actuator-space PD 및 torque clip
├─ tau_q = J^-T * tau_psi

RoK4FlatEnvCfg_PLAY
├─ 상속: RoK4FlatEnvCfg
├─ 재생 사용: ROK4_TEST_CFG
├─ UI 사용: RoK4PushTestWindow

RoK4RewardsCfg
├─ 상속: IsaacLab RewardsCfg
├─ 부모 reward terms 상속
├─ RoK4 reward terms 정의/override

RoK4TerminationsCfg
├─ 상속: IsaacLab TerminationsCfg
├─ time_out 상속 유지
├─ base_contact 비활성화
├─ illegal_body_contact 추가

RoK4FlatPPORunnerCfg
├─ RSL-RL PPO runner/network/algorithm 설정

RoK40nPolicyRunner
├─ RoK4PPO 사용
├─ upstream PPO update/learning-rate 동작 유지
├─ iteration 평균 KL -> Loss/kl
├─ iteration 최대 KL -> Loss/kl_max
├─ 별도 estimator MSE optimizer/checkpoint
├─ base velocity RMSE -> Metrics/estimator/*

RewardTermCfg(func=mdp.xxx)
├─ RoK4 로컬 mdp reward 함수 호출
├─ Isaac Lab 공통 mdp reward 함수 호출
├─ locomotion velocity 전용 mdp reward 함수 호출

```

Observation config의 새 정의와 field 교체

RoK40bservationsCfg 자체는 Isaac Lab 부모 ObservationsCfg 를 상속하지 않는다. RoK4에서 필요한 term만 모아 새로 만든 outer config container다. 반면 내부 RoK40bservationsCfg.PolicyCfg 는 ObsGroup, 즉 Isaac Lab ObservationGroupCfg 를 상속하므로 history, corruption, term concatenation 같은 observation-group 기능을 사용한다.

```

새 class 정의
RoK4ObservationsCfg                                # 부모 ObservationsCfg 상속 없음
├ PolicyCfg(ObsGroup)                             # actor용 5-step noisy history
├ CriticCfg(ObsGroup)                             # critic용 5-step clean history
└ PrivilegedCfg(ObsGroup)                         # critic 전용 current base/Foot state

환경 field 교체
LocomotionVelocityRoughEnvCfg
└ observations = 부모 observation config

RoK4FlatEnvCfg(LocomotionVelocityRoughEnvCfg)
└ observations: RoK4ObservationsCfg = RoK4ObservationsCfg()
  -> 상속받은 observations field의 객체를 RoK4 config로 통째로 교체

```

따라서 이것은 RoK4ObservationsCfg 가 부모 observation class를 상속해 method를 override하는 구조가 아니다. RoK4FlatEnvCfg 가 상속받은 observations config field를 새 객체로 대체하는 config-level override다. 부모의 부모의 base_lin_vel, joint_pos, joint_vel, height_scan term은 자동으로 남지 않는다. RoK4는 아래 6개 proprioceptive term을 actor와 critic용으로 각각 명시하고, simulator current state 10개를 별도의 critic-only privileged group으로 추가한다.

단일 frame term	차원	의미
base_ang_vel	3	base angular velocity
projected_gravity	3	base frame에 투영된 gravity direction
velocity_commands	3	[vx, vy, wz] command
actuator_pos	13	$J^{-1}(q - q_{\text{default}})$ actuator position relative to the default pose
actuator_vel	13	$J^{-1}(\dot{q} - \dot{q}_{\text{default}})$ actuator velocity relative to the default velocity
actions	13	이전 clipped raw actuator action
합계	48	history 적용 전 한 frame의 observation dimension

PolicyCfg 와 CriticCfg 모두 history_length=5 와 flatten_history_dim=True 를 적용하므로 각각 $48 * 5 = 240$ 차원이다. Policy만 enable_corruption=True 이므로 term별 additive noise가 history에 저장된다. Critic은 같은 물리량을 별도로 계산하지만 noise를 선언하지 않고 enable_corruption=False 로 두어 clean history를 만든다. 여기서 clean은 physics DR이 없는 상태가 아니라 simulator의 randomized 실제 state에 인위적 센서 noise만 추가하지 않는다는 뜻이다.

PrivilegedCfg 는 history 없이 아래 current-frame 10개를 제공한다.

Current privileged layout

index	term	차원	의미
0:3	base_lin_vel	3	body-frame root linear velocity [vx, vy, vz]
3:4	base_height	1	root world Z에서 environment origin Z를 뺀 평지 기준 높이
4:6	foot_height	2	Left/Right Foot_Link 원점 world Z에서 environment origin Z를 뺀 값
6:8	foot_contact	2	Left/Right current_contact_time > 0 binary flag
8:10	foot_air_time	2	Left/Right current uninterrupted air time [s]

Foot height는 ray-based height scanner나 collision-point query가 아니다. 현재 평지에서는 Foot_Link 원점이 발바닥 중심보다 약 0.004 m 위라는 고정 offset을 포함한 clearance proxy다. 이 4 mm는 현재 clearance reward의 std=0.04 m 보다 충분히 작으므로 별도 보정하지 않는다.

PPO runner의 obs_groups 는 actor에 ["policy"], critic에 ["critic", "privileged"] 를 연결한다. 따라서 외부 actor observation과 ONNX 입력은 기존과 같은 240차원이고 critic 입력은 $240 + 10 = 250$ 차원이다. yunho/concurrent-state-estimator 에서는 normalized policy history에서 command 30:45 를 제외한 225D로 3D base velocity를 추정하고 이를 detach해 240D history와 합친 243D를 내부 action MLP에 넣는다. Privileged 정보와 clean critic history 자체는 inference/ONNX 입력에 포함되지 않는다. ONNX는 하나의 16D tensor가 아니라 actions [1,13] 와 Actor

내부의 동일한 `estimated_base_lin_vel_b [1,3]` 를 별도 named output으로 제공한다. 상세 구조와 Sim2Sim/Sim2Real logging 방법은 estimator 전용 문서를 참조한다.

현재 `actuator_pos` 와 `actuator_vel` term은 부모 Isaac Lab의 `joint_pos_rel`, `joint_vel_rel` 패턴처럼 default state를 뺀 뒤 `actuator` 좌표로 변환한다. Position은 gait-ready default pose에서 0이 되고, 현재 `q_dot_default=0` 이므로 velocity 값은 absolute velocity와 수치상 같다. 별도의 수동 observation scale은 곱하지 않으며, relative 표현은 원점 이동이고 scale은 값의 크기 조절이라는 서로 다른 연산이다.

각 파일의 관계를 역할로 나누면 다음과 같다.

파일	관계	역할
<code>config/rok4/__init__.py</code>	task 등록	Train, Play, Teleop task 이름을 각각의 RoK4 env cfg와 RoK4FlatPPORunnerCfg 에 연결
<code>flat_env_cfg.py</code>	환경 main config	RoK4 scene, terrain, action, observation, reward, command, termination 설정
<code>domain_randomization_cfg.py</code>	DR config	RoK4 foot material, base mass, COM, external wrench, reset randomization 범위와 event mode 설정
<code>contact_force_visualizer.py</code>	ContactSensor 확장	env-0 좌우 발의 world-frame GRF vector를 화살표로 그리고 force magnitude를 확대된 숫자와 simulation-time X축 live plot으로 표시
<code>mdp/__init__.py</code>	mdp re-export	Isaac Lab locomotion mdp를 다시 export하고 RoK4 로컬 mdp 함수를 함께 노출
<code>mdp/actions.py</code>	RoK4 action 계산	13차원 clipped raw actuator action을 actuator target과 ADAPT joint target으로 변환
<code>mdp/commands.py</code>	RoK4 command 계산	90/5/5 episode role과 환경별 비동기 정지 구간을 uniform command에 추가
<code>mdp/observations.py</code>	RoK4 observation 계산	articulation joint position/velocity를 actuator position/velocity로 변환
<code>mdp/rewards.py</code>	RoK4 reward 계산식	RoK4 전용 actuator torque/velocity/acceleration, actuator limit, action smoothness reward 계산
<code>mdp/symmetry.py</code>	PPO data augmentation callback	policy history, privileged critic state, raw actuator action을 원본+좌우 mirror batch로 확장
<code>agents/rsl_rl_ppo_cfg.py</code>	학습 config	RSL-RL PPO network와 algorithm hyperparameter 설정
<code>scripts/rsl_rl/rok4_ppo.py</code>	학습 algorithm 확장	KL logging, 225D base-velocity estimator, 별도 MSE optimizer, RMSE/checkpoint와 fused export 제공
<code>assets/robots/rok4.py</code>	robot asset config	USD path, initial pose, actuator, joint order, action scale, self-collision 설정 제공
<code>assets/robots/rok4_adapt.py</code>	transmission/actuator 구현	configurable ADAPT 링크 길이, q <-> psi 행렬, actuator PD, torque mapping과 limit 적용
<code>scripts/check_rok4_zero.py</code>	actuator 정적 검사	passive, zero-command, torque-hold 상태에서 asset과 PD 동작 확인
<code>scripts/check_rok4_random.py</code>	actuator 동적 검사	작은 sinusoidal position target으로 전체 actuator 응답 확인
<code>scripts/check_rok4_joint_monkey.py</code>	joint/actuator 검사	joint 순서, axis, limit, teleport, ADAPT torque-PD sweep 확인

파일	관계	역할
IsaacLab velocity_env_cfg.py	부모 config	manager-based velocity locomotion의 공통 scene/action/obs/reward/termination 를 제공
IsaacLab mdp/rewards.py	reward 함수 모음	Isaac Lab 공통 reward 함수와 locomotion velocity 전용 reward 함수를 mdp.xxx namespace로 제공

한 줄로 요약하면, flat_env_cfg.py 는 RoK4 환경의 중심 설정 파일이고, __init__.py 는 task 이름을 등록하는 입구, rsl_rl_ppo_cfg.py 는 학습기 설정, rok4.py 는 로봇 asset 설정이다.

새로 생성된 파일

source/rok4_tasks/rok4_tasks/manager_based/__init__.py

Manager-based task 패키지 시작점이다. Isaac Lab의 ManagerBasedRLEnv 스타일 task를 RoK4 저장소 안에 넣기 위한 디렉터리 계층이다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/__init__.py

Locomotion task 그룹 시작점이다. 향후 velocity walking 외에 balancing, standing, whole-body tracking 같은 locomotion 계열 task를 추가할 때 같은 계층 아래에 둘 수 있다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/__init__.py

Velocity tracking task 그룹 시작점이다. 현재 RoK4의 첫 학습 목표가 command velocity를 따라 걷는 것이므로 이 계층을 만들었다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/__init__.py

Velocity task 안에서 robot별 config를 묶기 위한 계층이다. 지금은 rok4 만 있지만, 나중에 RoK4 variant가 늘어나면 같은 구조 아래에 추가할 수 있다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/rok4/__init__.py

Gymnasium task를 등록하는 파일이다. 여기에서 다음 task 이름이 Isaac Lab registry에 올라간다.

```
RoK4-Isaac-Velocity-Flat-v0
RoK4-Isaac-Velocity-Flat-Play-v0
RoK4-Isaac-Velocity-Flat-Teleop-v0
```

각 task는 isaacclab.envs.ManagerBasedRLEnv 를 entry point로 사용한다. 환경 설정은 flat_env_cfg.py 에서, RSL-RL PPO 설정은 agents/rsl_rl_ppo_cfg.py 에서 불러온다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/rok4/flat_env_cfg.py

이번 작업의 핵심 파일이다. RoK4 flat walking 환경을 정의한다.

주요 역할은 다음과 같다.

항목	내용
Scene	학습 task는 ROK4_TRAIN_CFG 를 {ENV_REGEX_NS}/Robot 위치에 spawn
Terrain	plane terrain 사용, rough terrain generator와 height scanner 제거
Action	ROK4_JOINT_ORDER 기준 13차원 normalized actuator position offset
Observation	actuator position/velocity를 포함한 blind proprioceptive history observation
Command	mixed/standing/walking/x/fast_forward/y/yaw/x_yaw 8개 episode role과 환경별 비동기 periodic freeze
Domain randomization	domain_randomization_cfg.py 의 apply_rok4_domain_randomization(self) 호출

항목	내용
Reward	velocity tracking, upright, action smoothness, 전체 13관절의 실제/목표 soft position limit, torque/acc/contact, no-jumps 및 signed anti-cross reward. foot-flat orientation과 목표 stance-width term은 현재 None
Termination	부모 timeout 유지, Foot를 제외한 모든 body의 illegal contact
Play cfg	ROK4_TEST_CFG 로 교체, 적은 env 수, corruption off, 자동 push/random external force off, 수동 Push UI 사용
Teleop cfg	Play cfg를 상속하고 1개 env, 600초 timeout, heading off, 자동 command resampling off 설정

RoK4 action space는 다음 13축이다.

```

Left leg:
  L_Hip_Yaw_Joint
  L_Hip_Roll_Joint
  L_Hip_Pitch_Joint
  L_Knee_Pitch_Joint
  L_Ankle_Pitch_Joint
  L_Ankle_Roll_Joint

Right leg:
  R_Hip_Yaw_Joint
  R_Hip_Roll_Joint
  R_Hip_Pitch_Joint
  R_Knee_Pitch_Joint
  R_Ankle_Pitch_Joint
  R_Ankle_Roll_Joint

Torso:
  Torso_Yaw_Joint

```

정책/ADAPT 순서는 반드시 rok4.py 의 ROK4_JOINT_ORDER 를 따른다. USD 내부 joint storage 순서는 좌우 관절이 교차되어 있으므로 RoK4AdaptActuator 가 canonical ADAPT 순서와 USD 순서를 내부에서 변환한다.

Action pipeline과 clip

RoK4 policy가 출력하는 13차원 값은 normalized actuator action이다. wrapper와 local action term에서 [-1, 1] 로 제한하고 actuator scale과 default actuator pose를 적용한 뒤 ADAPT 행렬로 joint target을 만든다.

```

actor network output
-> RslRlVecEnvWrapper clip_actions = 1.0
-> clipped_raw_actuator_action in [-1, 1]
-> psi_default = J^-1 * q_default
-> psi_target = psi_default + clipped_raw_actuator_action * ROK4_ACTUATOR_ACTION_SCALE
-> q_target = J * psi_target
-> tau_psi = Kp * (psi_target - psi) - Kd * psi_dot
-> tau_psi actuator torque-limit clip (maximum * 0.9)
-> tau_q = J^-T * tau_psi
-> PhysX joint effort (unscaled mechanical joint-torque limit으로 최종 보호)

```

actions.py 가 q_target 을 set_joint_position_target() 으로 기록하는 것은 Isaac Lab explicit actuator에 목표값을 전달하기 위한 interface다. 이 값이 PhysX position drive에 직접 입력되는 것은 아니다. 매 physics step에 Articulation이 target buffer와 현재 joint state를 RoK4AdaptActuator.compute() 에 전달하면 다음 과정이 실행된다.

```

actions.py
q_target을 joint-position-target buffer에 기록
-> Isaac Lab Articulation._apply_actuator_model()
-> RoK4AdaptActuator.compute(q_target, q, q_dot)
    | q_target DelayBuffer: 2 physics steps = 4 ms
    | delayed q_target, q, q_dot -> psi_target, psi, psi_dot
    | actuator PD로 tau_psi 계산
    | actuator 최대 torque * 0.9로 tau_psi clip
    | tau_q = J^-T tau_psi
    | position/velocity target 제거 + joint effort만 반환
-> PhysX에는 tau_q joint effort가 입력됨

```

실측 actuator command-path 지연은 ROK4_ACTUATOR_COMMAND_DELAY_STEPS=2 로 고정한다. 현재 physics period 0.002 s 에서 4 ms 이며 DelayedPDActuator 가 생성한 position/velocity/effort command buffer와 reset을 그대로 사용한다. 현재 task의 실질적인 명령은 position target이며 velocity/effort target은 zero tensor이다. RoK4는 부모의 joint-space PD compute() 를 호출하지 않고, 지연된 target을 ADAPT-space PD에 넣는 자신의 compute() 를 사용한다. 현재 q, q_dot feedback과 Actor observation은 지연하지 않는다. Environment reset 시 target history를 비우고 history가 부족한 초기 step 은 가장 최근 target을 반환하므로 임의의 zero target이 삽입되지 않는다.

따라서 policy action의 의미는 actuator position target이지만 simulation에 최종 입력되는 제어량은 ADAPT 변환을 거친 joint torque다. PhysX의 effort_limit_sim 은 joint mechanical maximum [150, 150, 300, 480, 180, 180] N m 와 torso 150 N m 를 factor 없이 사용한다. 이는 마지막 solver 안전 상한이며 actuator torque에 다시 0.9를 곱하는 제어 단계가 아니다.

중요한 점은 last_action observation이 q_target 이나 scaled actuator offset이 아니라 clip된 raw actuator action이라는 점 이다. motor target만 scale, default actuator pose, ADAPT mapping을 거친다.

Warning

action은 13차원, history observation은 240차원으로 이전 joint-space baseline과 크기가 같지만 각 원소의 의미가 actuator 좌표로 바뀌었다. 따라서 2026-07-15_17-28-41 Yunho v1을 포함한 기존 joint-space checkpoint를 이 브랜치에서 resume/play하지 말고 actuator-interface 학습을 새로 시작해야 한다.

action_rate_l2 와 second_action_rate_l2 reward는 observation의 last_action 과 같은 clipped_raw_action 좌표에서 각각 1차와 2차 차분을 계산하고 weight -0.01, -0.005 를 사용한다. Command role 비율도 검증 기준인 mixed=0.35, standing=0.05 를 유지한다. 현재 실험은 두 smoothness 가중치와 환경 비율을 고정하고 exact-zero standing default-pose weight만 -0.2 에서 -0.05 로 낮춘 단일 변수 비교다. Reward 함수 자체는 clamp하지 않지만 RoK4 RSL-RL runner의 clip_actions=1.0 이 ActionManager 이전에 정책 출력을 제한한다. Action scale은 actuator target 생성에만 사용하며 두 smoothness reward에는 적용하지 않는다. 다만 Hip Pitch/Knee action index 2, 3, 8, 9 는 RoK4의 동적 보행 자유도를 위해 squared-error weight를 0.5 로 완화한다.

ROK4_ACTUATOR_ACTION_SCALE 은 기존 Isaac Gym RoK4의 actuator action scale과 동일하다. 좌측 다리, 우측 다리, torso 순서의 값은 다음과 같다.

```
[0.4, 0.5, 1.25, 1.5, 0.75, 0.75,
0.4, 0.5, 1.25, 1.5, 0.75, 0.75,
0.4]
```

이 scale 배열은 actuator target 계산에 사용한다. 두 action smoothness reward는 raw action을 사용한다.

ADAPT 링크/limit 설정

RoK4AdaptTransmission 은 link_alpha=0.09845 m, link_beta=0.06 m 를 받아 ratio=beta/alpha 와 J, J⁻¹, J^T, J^{-T} 를 만든다. 링크 길이가 바뀌면 rok4.py 의 두 상수만 수정하면 action, observation, actuator PD, torque mapping이 같은 행렬을 사용한다.

Actuator 최대 torque는 한쪽 다리 [150, 150, 150, 150, 90, 90] N m, torso 150 N m 이고 velocity는 한쪽 다리 [12, 12, 12, 12, 15, 15] rad/s, torso 12 rad/s 이다. torque_limit_factor=0.9 와 velocity_limit_factor=0.9 를 각각 곱한 값만 실제 actuator limit으로 저장한다. Torque limit은 PD torque clip과 torque-limit reward에 사용하고, velocity limit은 velocity-limit reward에 사용한다. 별도의 actuator position soft limit은 추가하지 않는다. 변환된 joint torque에 대한 PhysX effort_limit_sim 은 factor를 적용하지 않은 joint mechanical maximum을 사용한다.

학습 command 범위

Flat 학습은 lin_vel_x=(-0.3, 0.85) m/s, lin_vel_y=(-0.3, 0.3) m/s, ang_vel_z=(-0.6, 0.6) rad/s 를 사용한다. 이전 Isaac Gym RoK4의 후진 하한을 복원하고, 전체 범위를 표본화하는 mixed population과 축별 command population을 함께 사용한다.

RoK4CommandsCfg 는 부모 CommandsCfg 의 base_velocity 를 RoK4PeriodicFreezeVelocityCommandCfg 로 교체한다. 일 반 command resampling interval은 부모와 같은 고정 (10.0, 10.0) s 를 사용한다. 각 episode reset에서 환경은 확률적으로 다음 역할 중 하나를 받고, 해당 episode 동안 역할을 유지한다.

35% mixed	: vx, vy, wz 전체 표본화
5% standing	: episode 전체 exact-zero command
5% walking	: freeze 없는 mixed moving command, planar norm >= 0.10 m/s
20% x	: 저속 대칭 전후진, vx=+/- [0.10, 0.30] m/s
5% fast_forward	: 고속 전진, vx=[0.30, 0.85] m/s
10% y	: lateral 좌/우 이동, vy만 활성화
10% yaw	: 시계/반시계 회전, wz만 활성화
10% x_yaw	: 저속 대칭 전후진 + 회전, vy=0

코드의 역할 이름은 command vector의 축에 직접 대응하도록 x , y , yaw , x_yaw 를 사용한다. 문서의 보행 의미로는 x 가 sagittal 전후진, y 가 lateral 좌우 이동이다. x , y , yaw 내부의 양/음 부호는 각각 50:50 확률로 표본화한다. x 와 x_yaw 의 vx 는 $\pm$ Uniform(0.10, 0.30) m/s 로 제한하여 +0.3 과 -0.3 m/s 가 같은 전용 학습 범위에 놓이게 한다. x_yaw 는 vx 와 wz 부호를 독립적으로 표본화하므로 전진/후진과 시계/반시계 회전의 네 조합을 만든다. 별도 fast_forward 역할은 Uniform(0.30, 0.85) m/s 로 기존 고속 전진 능력을 유지한다. vy 의 최소 절댓값은 0.10 m/s, wz 는 0.10 rad/s 이다.

mixed, x , fast_forward, y , yaw , x_yaw 환경은 10.0 s cycle 안에서 서로 다른 random phase로 시작한다. 각 환경은 독립적으로 Uniform(1.5, 3.0) s standing duration을 표본화하므로 모든 환경이 동시에 멈추지 않는다. 이 여섯 역할의 비율 합은 90%이므로 해당 population에서 moving-to-standing transition을 학습한다. standing 은 항상 exact zero이고, walking 은 freeze 없이 planar command norm이 0.10 m/s 이상일 때까지 다시 표본화한다. 평균 freeze duration 2.25 s 를 기준으로 하면 전체 time sample은 대략 standing 25%, moving 75%가 된다.

Episode 중 일반 10.0 s command resampling과 freeze 종료 resampling은 현재 역할을 유지한 채 그 역할 안에서 새 command를 만든다. 환경이 reset되면 역할 자체를 위 비율로 다시 무작위 표본화한다. 따라서 Gym의 축별 command 종류는 보존하지만, 특정 environment index를 한 역할에 영구 고정하지는 않는다.

RoK4의 저수준 command interface는 기존 Gym, Teleop, ROS cmd_vel 과 동일한 base-frame [vx , vy , wz] 이다. 따라서 heading_command=False, rel_heading_envs=0.0, heading=None 을 사용하고 vx , vy , wz 를 각 범위에서 직접 표본화한다. World target heading이나 heading-error controller는 학습 command 생성에 사용하지 않는다. 이전 rel_standing_envs=0.05 부모 경로는 역할 기반 standing과의 중복 배정을 피하기 위해 현재 rel_standing_envs=0.0 으로 끈다. Exact-zero command는 periodic freeze window와 standing 역할에서 생성하며, 두 경로 모두 command를 [0, 0, 0] 으로 만드는 동시에 is_standing_env=True 를 설정한다. 일반 mixed moving command는 크기가 작더라도 standing으로 변환하지 않으므로 연속적인 저속 command 범위를 유지한다.

모든 exact-zero standing에서 weight -0.05 인 stand_still_joint_deviation_l1 이 command term의 is_standing_env mask를 사용하여 전체 13관절의 실제 joint position을 default joint pose 근처로 유도한다. 이는 검증 기준 -0.2 의 25% 강도다. 양발 standing 편향은 남기되 외란이나 급정지 중 recovery step과의 경쟁을 줄이는 단일 변수 실험이며, 작은 non-zero 이동 command에는 적용하지 않는다.

Episode timeout, periodic freeze, push timer 관계

학습 환경에는 서로 독립적인 세 종류의 시간이 있다.

시간 상태	환경별 여부	reset 시 동작	현재 범위
Episode elapsed time	환경별	해당 환경만 0으로 초기화	timeout 20 s = policy step 2,000 개
Periodic freeze phase	환경별	Uniform(0, 10) s 로 새 phase 표본화	cycle 10 s, duration Uniform(1.5, 3.0) s
Training push time-left	환경별	Uniform(10, 15) s 로 다음 push 시간 표본화	world-frame vx/vy 에 각각 Uniform(-0.5, 0.5) m/s 추가

Episode timeout은 ManagerBasedRLEnv 의 episode_length_buf 로 환경마다 관리한다. 학습 시작 직후에는 모든 환경이 episode step 0에서 출발하므로, early termination이 전혀 없는 환경들은 global simulation time 20 s, 40 s 처럼 timeout이 서로 맞을 수 있다. 반면 환경 A가 global time 3 s 에 illegal contact로 종료되면 A의 episode counter만 0으로 돌아가며, 다음 episode를 끝까지 생존할 경우 global time 약 23 s 에 timeout된다. 즉 초기 episode phase를 명시적으로 randomize하지는 않지만, 환경별 early termination과 reset이 진행되면서 episode phase가 자연스럽게 비동기화된다.

Periodic freeze는 episode elapsed time과 별도인 command-term phase다. 해당 환경이 reset될 때 phase를 [0, 10) s 에서 독립적으로 다시 뽑고, phase < duration 이면 exact-zero freeze를 적용한다. 따라서 reset 후 반드시 10초를 기다렸다가 멈추는 구조가 아니며, 표본화된 phase가 duration보다 작으면 reset 직후부터 freeze 상태일 수도 있다. Phase가 10초를 넘으면 0으로 wrap하고 새 duration을 표본화한다. Freeze를 빠져나오는 순간에는 현재 episode role을 유지한 채 새 이동 command를 표본화한다.

Training push_robot 은 Isaac Lab 부모 locomotion task에서 상속한 interval event다. is_global_time=False 이므로 모든 환경에 동시에 push하는 global timer가 아니라 각 환경이 별도의 time-left를 갖는다. Reset마다 그 환경의 다음 push 시간이 10~15 s 로 다시 표본화되고, push가 발생한 뒤에도 같은 범위로 다음 interval을 뽑는다. Timeout이 20 s 이므로 끝까지 생존한 episode에는 보통 후반부에 push 한 번이 들어가고, 10초 전에 종료된 episode에는 push가 없을 수 있다. 첫 push 뒤 다음 push까지 최소 10초가 필요하므로 정상적인 20초 episode 안에서 두 번째 push가 발생하는 경우는 사실상 없다.

Freeze phase와 push time-left는 서로 독립적으로 표본화된다. 따라서 push는 이동 command 중, exact-zero freeze 중, 또는 freeze 진입/종료 부근 어느 시점에도 들어올 수 있다. 이 조합 덕분에 policy는 같은 외란을 다양한 command 상태에서 복원하게 된다. Play와 Teleop은 자동 push_robot event를 끄며, 화면의 RoK4 Push Test 버튼이 별도의 수동 외란 경로를 제공한다.

```

global t=0 s : 모든 env episode counter=0
               env A freeze phase=8.7 s, next push=12.4 s
               env B freeze phase=1.0 s, next push=14.1 s
global t=3 s : env A early termination -> A counter=0, freeze phase와 push timer 재표본화
global t=20 s : 생존한 env B timeout/reset
global t=23 s : env A가 3 s 이후 20 s 생존했다면 timeout/reset

```

RoK4-local feet_air_time_touchdown_biped 함수는 target_air_time=0.50 s, weight=2.0 을 사용한다. Contact sensor의 last_air_time 을 읽어 정확히 한 발이 first contact가 된 step에만 last_air_time - 0.50 을 한 번 지급한다. Swing 중, 계속된 지지, 양발 동시 first contact, planar command norm 0.05 m/s 이하에서는 0이다. 0.50 s 보다 짧은 완료 swing에는 음수, 그보다 긴 완료 swing에는 양수를 반환한다. 최대 보상 air-time cap은 없으므로 긴 single support는 velocity tracking과 no_jumps 등 다른 gait term이 함께 제한한다. Event 기반 값이므로 이전 dense, squared 또는 capped touchdown air-time의 TensorBoard 크기와 직접 비교하지 않는다.

현재 활성 air-time term은 stateful FeetAirTimeTouchdownBiped class다. Reward와 동일한 single-touchdown 및 moving-command mask로 완료된 last_air_time 을 누적하여 아래 물리 단위 metric으로 기록한다. Aggregation이 필요 없는 비교 실험을 위해 stateless feet_air_time_touchdown_biped 함수도 남긴다. Reward Manager의 Episode_Reward/feet_air_time 은 weighted reward sum이므로 실제 평균 air time과 같지 않다.

Policy interval이 0.01 s 이므로 touchdown error의 event 기울기는 $2.0 * 0.01 = 0.02$ 다. 이는 K1의 weight=1.0, policy interval 0.02 s 와 같은 기울기지만, RoK4는 더 긴 0.50 s zero crossing, 양발 동시 touchdown 제외, 별도 no_jumps penalty를 유지한다.

no_jumps 는 Isaac Lab 공통 mdp.desired_contacts 를 weight=-2.0 과 force threshold=1.0 N 으로 사용한다. 좌우 Foot 의 최근 5개 contact-force sample 중 어느 쪽에도 threshold를 넘는 접촉이 없을 때만 raw penalty 1 을 반환한다. 따라서 정상 single stance와 toe-off는 허용하고 양발이 동시에 뜬 flight phase만 억제한다. 두 velocity-tracking term의 최대 pre-dt 합 +2.0 을 flight 구간에서 상쇄하는 첫 실험값이며, 좌우 교대 순서나 한 발 지지의 최대 시간을 직접 강제하지 않는다.

feet_touchdown_acc 함수는 ROBOTIS K1의 event penalty와 동일하게 first-contact 발의 world-frame 선형가속도 초과량을 계산하지만, 현재 reward term은 None 이다. touchdownacc50 실험에서는 threshold=50 m/s², weight=-0.002 를 사용했으나 MuJoCo와 실기에서 광 찍는 착지가 유지되었다. compute_first_contact(step_dt) 는 최근 10 ms 내 접촉을 검출하지만 reward가 읽는 body_lin_acc_w 는 5번째 physics substep 이후의 최신 가속도이므로, 앞선 2 ms substep에서 발생한 충격 peak와 시간적으로 어긋날 수 있다. 함수와 단위 테스트는 비교용으로 남긴다. 현재 soft-landing 실험은 stateful FeetTouchdownVelocityL2 를 활성화한다. 이전 policy step에서 공중이었던 발의 world-Z 속도를 저장하고, first contact가 발생하면 $\text{relu}(-v_z_{\text{prev}})^2$ 를 사건당 한 번 계산해 weight -10.0 을 적용한다. 접근 중과 계속된 stance에는 영향을 주지 않으며 reset 직후 이전 sample이 없는 초기 접촉도 제외한다. FeetContactForceL2 class는 first contact부터 0.10 s 동안 filtered world-frame 접촉 합력 $||[F_x, F_y, F_z]||$ 의 최대값을 누적한다. 현재 shaping config는 feet_contact_force=None 이므로 GRF가 학습을 shaping하지 않는다. 대신 feet_contact_force_metrics 가 metric_only=True 로 같은 peak를 TensorBoard Metrics/feet_touchdown/mean_peak_normal_force 에 기록하면서 reward에는 항상 0을 반환한다. Contact Forces debug visualization도 그대로 사용할 수 있다.

검증된 기준 checkpoint는

2026-08-12_23-45-39_privileged250_gain240_160_80_air050_w2_tdvel10_ar01_ar2_005_noforce_delay4ms_fresh20k/model_다. 이 checkpoint는 평균 착지 직전 하강속도를 약 0.044 m/s 로 낮추면서 velocity tracking과 평균 완료 air time을 유지했다. Log/checkpoint는 Isaac Lab log directory에 보존하고 Git에는 코드와 설정만 기록한다. 기존 Gym 호환 continuous feet_contact_velocity_l2 함수는 비교용으로 남아 있지만 Reward Manager term은 None 이다.

Concurrent estimator를 포함한 최신 MuJoCo Sim2Sim 기준은 commit df37f44 의

2026-08-21_02-28-25_concurrent_estimator_stand005_fresh25k/model_24999.pt 이다. 이 checkpoint는 외부 240D observation 계약을 유지하면서 ONNX에서 actions [1,13] 와 estimated_base_lin_vel_b [1,3] 를 분리해 출력한다. Isaac Sim Teleop과 MuJoCo Sim2Sim에서 방향별 보행, 양발 정지, 급정지 및 push recovery 동작을 확인했다. 상세한 estimator graph, supervised MSE 학습, export 및 검증 경계는 rok4_concurrent_state_estimator_ko.rst 를 기준으로 한다.

두 활성 stateful term은 각각 air time과 착지 직전 속도의 episode 합과 touchdown 횟수를 GPU tensor로 누적하고 environment reset 시 다음 event-weighted 평균을 TensorBoard에 기록한다.

- Metrics/feet_touchdown/mean_pre_touchdown_vertical_speed: 직전 airborne sample의 평균 하강속도 크기 [m/s]
- Metrics/feet_touchdown/mean_air_time: touchdown에서 완료된 평균 last_air_time [s]

두 값은 reward 가중치나 dt 가 적용되지 않은 물리량이며 매 step CPU 동기화를 만들지 않는다.

Play 환경은 현재 standing 검증을 위해 lin_vel_x=0.0 m/s, lin_vel_y=0.0 m/s, ang_vel_z=0.0 rad/s 로 고정한다. Teleop 환경은 자동 표본화를 끄고 사용자가 입력한 base-frame [lin_vel_x, lin_vel_y, ang_vel_z] 를 command buffer 에 직접 기록한다. Play와 이를 상속하는 Teleop은 학습 전용 episode role과 periodic freeze를 비활성화하므로 수동/고정 command를 덮어쓰지 않는다. 두 환경 모두 RoK4PushTestWindow 를 사용하므로 선택 환경의 command와 실제 vx, vy, vz,

wz, |vxy| 를 숫자로 비교하면서 고정 command와 수동 command 각각에서 외란 복원 능력을 같은 버튼으로 검사할 수 있다.

초기 root 위치

기존 Gym 메모에서 gait-ready CoM 기준 (0.0575 / 2, 0, 0.835) m 는 gait-ready base 기준 (0.0552, 0, 0.907) m 에 대응한다. 다리를 펴고 선 자세의 base 높이는 z=0.919 m 이다. ROK4_TRAIN_CFG 의 실제 articulation root 초기 위치는 (0.0552, 0.0, 0.929) m 이며, 이는 다리를 편 직립 base 높이보다 0.010 m 위에 로봇을 살짝 띄워 배치한 값이다.

이 pos 는 root의 world/environment 위치이고, 초기 관절 자세는 _ROK4_INIT_JOINT_POS 가 별도로 정의한다. 현재 _ROK4_INIT_JOINT_POS 도 Gym의 활성 defaultJointAngles 및 nominalJointAngles gait-ready 자세와 동일하게 맞춰져 있다.

Gym과 동일한 초기 관절 자세

관절 그룹	초기 각도 [rad]
Hip Yaw / Hip Roll	0.0
Hip Pitch	-0.0924
Knee Pitch	0.345
Ankle Pitch	-0.253
Ankle Roll / Torso Yaw	0.0

초기 관절 자세는 $\psi_{\text{default}} = J^{-1} * q_{\text{default}}$ 로 변환되어 actuator action의 기준 자세가 된다.

Isaac Lab의 train.py 와 play.py 를 사용할 때는 RslRlVecEnvWrapper 가 자동으로 clipping을 수행한다. 반면 ONNX/TorchScript policy를 직접 호출하는 sim2sim 또는 sim2real 코드에서는 같은 동작을 외부 코드에서 직접 수행해야 한다.

```
policy_output = policy(obs)
clipped_raw_action = torch.clamp(policy_output, -1.0, 1.0)
psi_target = default_actuator_pos + clipped_raw_action * ROK4_ACTUATOR_ACTION_SCALE
q_target = adapt.actuator_to_joint_position(psi_target)
last_action = clipped_raw_action
```

Actor observation은 5-step history를 사용한다.

```
base_ang_vel
projected_gravity
velocity_commands
actuator_pos_rel
actuator_vel_rel
last_action
```

Actor에는 다음 정보를 넣지 않는다.

```
camera image
terrain height scan
base linear velocity
```

따라서 현재 구조는 estimator가 없는 history MLP 방식의 blind walking baseline이다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/rok4/domain_randomization_

RoK4 flat task의 domain randomization 값을 따로 관리하는 파일이다. flat_env_cfg.py 안에 긴 DR block을 직접 두지 않고, 다음 한 줄로 적용한다.

```
apply_rok4_domain_randomization(self)
```

이 분리의 목적은 reward, observation, action 설정과 DR 튜닝값을 섞지 않는 것이다. 앞으로 friction, restitution, mass, COM, reset perturbation을 조정할 때 이 파일을 먼저 보면 된다.

Baseline commit d949d40 이후 변경 범위는 다음과 같다.

구분	baseline 대비 변경 여부	현재 내용
Policy observation noise	변경 없음	base_ang_vel +-0.2, projected_gravity +-0.05, actuator_pos +-0.01 rad, actuator_vel +-1.5 rad/s 를 유지

구분	baseline 대비 변경 여부	현재 내용
		하머 command와 last action에는 noise가 없다.
Physics DR	변경	correlated Foot material DR과 environment/joint별 static friction, viscous friction, armature scale DR 추가
Reset state	변경	joint position 0.9~1.1 scale은 유지하고 joint velocity를 고정 0에서 Uniform(-0.1, 0.1) rad/s 로 변경
Termination/timeout	변경 없음	Foot 이외 illegal contact 종료와 episode timeout 20 s 는 baseline 동작을 유지

현재 DR 구조는 다음과 같다.

그룹	mode	현재 설정
Robot/Foot physics material	startup	모든 robot collision shape를 nominal 0.8/0.6 으로 먼저 고정한 뒤, 환경별 좌우 Foot만 공통 static friction 0.5~0.9, dynamic 0.75 * static (0.375~0.675), restitution 0.1~0.3 으로 덮어쓴다.
Joint physics	startup	각 environment/joint의 nominal ROK4_STATIC_FRICTION, ROK4_VISCOUS_FRICTION, ROK4_ARMATURE 를 서로 독립적으로 Uniform(0.8, 1.2) 배 scale
Body mass	startup	base, upper, lower body group을 각각 0.9~1.25 배로 scale
Body COM	startup	base, upper, lower body group의 COM offset을 그룹별 범위로 randomize
External base wrench	reset	현재 force/torque range는 0으로 두어 외력은 꺼진 상태
Reset joint pose	reset	각 environment/joint가 default joint position을 독립적으로 0.9~1.1 scale 한다. Default가 0인 관절은 그대로 0 이다.
Reset joint velocity	reset	각 environment/joint가 절대 속도 Uniform(-0.1, 0.1) rad/s 를 독립적으로 표본화
Reset base pose/velocity	reset	episode reset마다 base x/y/yaw pose와 base velocity를 약하게 randomize
Root XY velocity push	interval	환경별 10~15 s timer로 world-frame root x/y velocity에 각각 -0.5~0.5 m/s 를 추가

startup DR은 scene 생성 시 각 environment에 대해 한 번 샘플링된다. reset DR은 해당 environment의 episode reset마다 다시 샘플링된다. interval event는 환경별 남은 시간을 가지며 reset 때 timer도 다시 표본화한다. 4096개 environment 학습에서 CPU-heavy DR을 매 reset 수행하지 않기 위해 material, mass, COM은 현재 startup 으로 둔다.

Material event는 먼저 G1/Digit 방식과 같은 nominal robot friction $\mu_{static}=0.8$, $\mu_{dynamic}=0.6$ 을 모든 robot collision shape에 명시한다. 그 뒤 RoK4 로컬 randomize_rigid_body_material_correlated event가 환경마다 Foot material bucket 하나를 선택해 좌우 Foot의 모든 collision shape에 공통 적용한다. Foot의 μ_{static} 은 Uniform(0.5, 0.9) 에서 표본화하고 $\mu_{dynamic} = 0.75 * \mu_{static}$ 으로 계산하므로 dynamic friction은 0.375~0.675 이며 항상 static보다 작다. 두 값을 독립 표본화하지 않아 비물리적인 $\mu_{dynamic} > \mu_{static}$ 조합을 막고, 좌우 발 마찰 차이가 policy 비대칭을 만드는 것도 피한다. 이 첫 범위는 nominal 0.8/0.6 주변의 미끄러운 조건부터 먼저 확장하는 단계다. Foot friction DR 범위는 학습 task에만 적용한다. Play와 Teleop은 Foot까지 nominal $\mu_{static}=0.8$, $\mu_{dynamic}=0.6$ 으로 고정한다. Foot 이외 shape의 restitution은 asset 원본값을 유지하고, Foot restitution만 학습에서 0.1~0.3 으로 덮어쓴다.

Joint physics DR은 Isaac Lab의 randomize_joint_parameters startup event를 사용한다. Isaac Sim 5에서는 friction_distribution_params=(0.8, 1.2) 가 nominal static friction과 viscous friction에 각각 독립적인 uniform scale

을 적용한다. `armature_distribution_params=(0.8, 1.2)` 는 별도의 독립 표본으로 joint armature를 scale한다. 따라서 environment `e` 와 joint `j` 에 대해 다음 관계가 성립한다.

$$\mu_{s,e,j} = \mu_{s,j}^{nominal} U_{s,e,j}(0.8, 1.2)$$

$$c_{v,e,j} = c_{v,j}^{nominal} U_{v,e,j}(0.8, 1.2)$$

$$I_{a,e,j} = I_{a,j}^{nominal} U_{a,e,j}(0.8, 1.2)$$

세 random variable은 서로 독립이며 4096개 environment에도 독립적으로 생성된다. 이 event는 PhysX joint property만 바꾸며 actuator-space PD의 `ROK4_ACTUATOR_KP` 와 `ROK4_ACTUATOR_KD` 는 변경하지 않는다. Play와 Teleop에서는 joint_physics event를 None 으로 제거하여 세 물성을 nominal 값으로 유지한다.

Joint reset은 RoK4 로컬 `reset_joints_by_position_scale_and_velocity` 를 사용한다. 기존 Isaac Lab `reset_joints_by_scale` 에 `velocity_range=(-0.1, 0.1)` 만 넣으면 default joint velocity 0 에 random scale을 곱하므로 결과가 계속 0이다. 로컬 함수는 position에는 기존처럼 default pose의 0.9~1.1 scale을 적용하지만, velocity에는 scale이 아닌 절대값 `Uniform(-0.1, 0.1) rad/s` 를 직접 표본화한다. 이 값은 episode reset마다 모든 environment와 13개 관절에 독립적으로 생성되고 joint velocity limit 안으로 clamp된다.

Mass와 COM DR은 다음 세 body group으로 나누어 관리한다.

그룹	링크	mass scale	COM range
base	Base_Link	0.9~1.1	x/y/z +-0.01 m
upper	Upper_Body_Link	0.9~1.25	x/y/z +-0.03 m
lower	left/right leg 전체 link와 foot link	0.9~1.25	x/y/z +-0.005 m

RoK4 control timing

RoK4는 Isaac Lab 본체의 `LocomotionVelocityRoughEnvCfg` 를 수정하지 않고, `RoK4FlatEnvCfg` 안에서 simulation/control timing만 override한다.

설정	값
sim.dt	0.002 s
physics frequency	500 Hz
decimation	5
policy/action period	0.010 s
policy/action frequency	100 Hz
contact sensor update period	0.002 s
contact-force history length	5 physics samples

Contact sensor의 `history_length` 는 Digit locomotion 설정과 같은 방식으로 `self.decimation` 에 맞춘다. 따라서 현재는 10 ms policy interval 동안 실행되는 5개의 2 ms physics step마다 contact-force sample 하나를 보존한다. 이 이력은 contact 기반 reward와 termination에서 짧은 접촉을 확인하기 위한 sensor buffer이며, actor 입력을 5 frame 쌓는 `self.observations.policy.history_length` 와는 서로 다른 설정이다.

source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/rok4/agents/rsl_rl_ppo_cfg

RSL-RL PPO runner 설정 파일이다.

현재 설정은 RoK4용 network/observation normalization 설정에 G1-style PPO algorithm 값을 섞은 flat walking 초기 baseline이다. 이 값들은 최종 튜닝값이 아니라 첫 RoK4 flat 보행 실험을 위한 시작점이다.

설정	값
experiment name	rok4_flat
max iterations	5000
steps per env	24
actor hidden dims	[512, 256, 128]
critic hidden dims	[512, 256, 128]
actor/critic obs normalization	True
action clipping	clip_actions = 1.0
activation	elu

설정	값
learning rate	1.0e-3
entropy coef	0.002
value loss coef	1.0
desired KL	0.01
obs groups	actor는 noisy policy 240차원, critic은 clean critic 240 + current privileged 10 = 250차원
symmetry data augmentation	original:left-right mirror = 1:1, mirror loss는 False

현재 actor는 blind noisy policy history만 사용하고, critic은 같은 항목의 clean history와 current base/Foot state를 받는 asymmetric 구조다. Privileged 10D에는 history를 적용하지 않는다. Explicit estimator는 아직 넣지 않았으며 rough terrain 단계에서 검토한다.

entropy_coef 는 exploration standard deviation을 키우는 방향의 entropy 항에 곱해지는 계수다. 현재 0.002 는 최초 0.008 과 저-noise 실험값 0.001 사이의 중간 설정이다. 최초 설정보다 특정 관절의 noise standard deviation이 과도하게 커지는 현상을 줄이면서, 0.001 에서 관찰된 강한 exploration 감소를 완화하기 위한 값이다.

desired_kl=0.01 은 old policy와 update 중인 new policy의 Gaussian action distribution 차이를 관리하는 adaptive learning-rate 기준이다. 현재 RSL-RL은 각 mini-batch KL이 0.02 보다 크면 learning rate를 1.5 로 나누고, $0 < KL < 0.005$ 이면 1.5 배하며, 그 사이에서는 유지한다.

RoK4 좌우 symmetry data augmentation

현재 branch는 Mittal et al. (2024)의 corrected on-policy PPO symmetry data augmentation을 RSL-RL이 제공하는 RslRlSymmetryCfg 경로로 사용한다. use_data_augmentation=True, use_mirror_loss=False 이므로 별도의 symmetry loss 항이나 mirror_loss_coeff 튜닝은 없다. 한 mini-batch의 원본 sample 수가 B 이면 compute_symmetric_states() 가 같은 network update 안에서 다음 순서의 2B sample을 만든다.

```
observations: [o_1, ..., o_B, L_g(o_1), ..., L_g(o_B)]
actions:      [a_1, ..., a_B, K_g(a_1), ..., K_g(a_B)]

mirrored half가 재사용하는 rollout 값:
    advantage A, return R, target value, old log probability
```

Actor와 critic network를 두 벌 만드는 방식이 아니다. 같은 actor/critic이 원본과 mirror sample을 모두 계산하며, network 입력 dimension은 actor 240, critic 250 으로 유지된다. 증가하는 것은 PPO update mini-batch의 sample 행 수다. 현재 actor/critic empirical observation normalization도 기존 baseline과 같이 True 로 유지하며 symmetry callback은 raw observation group을 먼저 반전한다.

Policy history의 실제 메모리 순서는 주의가 필요하다. Isaac Lab ObservationManager는 각 term의 5-step history를 먼저 flatten한 뒤 term들을 concatenate한다. 따라서 240차원 layout은 다음과 같은 term-major 구조다.

```
[base_ang_vel    5 x 3]  indices  0:15
[projected_gravity 5 x 3] indices 15:30
[velocity_command 5 x 3] indices 30:45
[actuator_pos     5 x 13] indices 45:110
[actuator_vel     5 x 13] indices 110:175
[last_action      5 x 13] indices 175:240
```

그러므로 기존 Gym의 48차원 frame을 5번 연속 배치한 것으로 보고 [0:48], [48:96] 처럼 자르면 잘못된 변환이 된다. symmetry.py 는 각 term을 [5, term_dim] 으로 복원해 모든 history sample에 공간 반전을 적용하며 시간 순서는 뒤집지 않는다.

좌우 반사면은 robot의 x-z plane, 즉 y -> -y 다. Vector term의 component 부호는 다음과 같다.

Observation vector mirror

term	원본	mirror
base angular velocity	[wx, wy, wz]	[-wx, wy, -wz]
projected gravity	[gx, gy, gz]	[gx, -gy, gz]
direct velocity command	[vx, vy, wz]	[vx, -vy, -wz]
critic privileged base linear velocity	[vx, vy, vz]	[vx, -vy, vz]
critic privileged base height	[h_base]	[h_base]
critic privileged Foot height/contact/air time	[L, R] for each bilateral term	[R, L] for each bilateral term

Actuator position, actuator velocity, last action, PPO action에는 모두 같은 13D actuator 변환 K_g 를 사용한다. Canonical 순서에서 식은 다음과 같다.

```
[L_HY, L_HR, L_psi1, L_psi2, L_psi3, L_psi4,
 R_HY, R_HR, R_psi1, R_psi2, R_psi3, R_psi4, Torso_Yaw]

K_g(x) =
[-R_HY, -R_HR, R_psi1, R_psi2, R_psi4, R_psi3,
 -L_HY, -L_HR, L_psi1, L_psi2, L_psi4, L_psi3, -Torso_Yaw]
```

마지막 actuator pair psi3/psi4 교환은 임의의 규칙이 아니다. ADAPT joint block의 좌우 반사를 $P_q = \text{diag}(1, 1, 1, -1)$ 로 두면 position transmission에 대해 정확히 다음 관계가 성립한다.

```
q = J psi
J P_psi = P_q J
```

즉 actuator의 마지막 두 좌표를 교환하면 joint hip-pitch, knee, ankle-pitch는 유지되고 ankle-roll만 반전된다. Hip-yaw, hip-roll, torso-yaw는 좌우 반사에서 부호가 바뀐다. 단위 검사는 이 행렬 관계, $K_g(K_g(x))=x$, $L_g(L_g(o))=o$, 그리고 TensorDict B -> 2B 확장을 확인한다.

기존 a2c_continuous.py 의 set_mirror_matrix() 는 mirror loss용이었지만 첫 48개 observation과 13개 action에 사용한 좌우 index/sign 규칙은 현재 변환과 일치한다. 이번 구현은 그 환경별 변환만 재사용하고, 최적화 방식은 mean-action MSE mirror loss가 아니라 RSL-RL의 PPO data augmentation이다. 이전 checkpoint를 이어 교정하지 않고 이 branch에서는 fresh run으로 비교한다.

scripts/rsl_rl/rok4_ppo.py

Isaac Lab 또는 설치된 rsl_rl 파일을 수정하지 않고 KL과 concurrent velocity estimator를 제공하는 RoK4 로컬 PPO/runner 확장이다. RoK4PPO 는 upstream PPO의 adaptive KL 계산값을 누적하고 estimator를 별도 Adam/MSE로 학습한다. RoK4OnPolicyRunner 는 이 PPO를 생성하고 estimator optimizer checkpoint와 physical RMSE를 관리한다. Estimator와 fused 240D input, 13D action/3D estimated-velocity ONNX output의 상세 수식은 rok4_concurrent_state_estimator_ko.rst 를 참조한다.

현재 num_learning_epochs=5, num_mini_batches=4 이므로 한 PPO iteration에는 20개의 mini-batch KL이 계산된다.

KL TensorBoard tags

tag	의미
Loss/kl	한 iteration의 20개 mini-batch KL 평균
Loss/kl_max	같은 iteration에서 관측된 최대 mini-batch KL
Loss/learning_rate	adaptive KL schedule 적용 후 learning rate

Loss/kl 은 PPO의 Loss/surrogate 와 다른 값이다. Surrogate는 actor 최적화 목적함수이고, KL은 old/new policy가 얼마나 달라졌는지를 측정한다. 기존 TensorBoard event 파일에는 새 tag가 소급 추가되지 않으며, 수정 후 RoK4 train.py 로 시작한 run부터 기록된다.

scripts/rsl_rl/_run_isaaclab_rsl.py

Isaac Lab 원본 scripts/reinforcement_learning/rsl_rl/train.py 와 play.py 를 수정하지 않고 RoK4 task와 선택적인 teleop 입력을 삽입하기 위한 wrapper이다.

동작 흐름은 다음과 같다.

1. rok4_lab/source/rok4_tasks 를 sys.path에 추가
2. 현재 작업 디렉터리의 Isaac Lab RSL-RL script 경로 찾기
3. Isaac Lab 원본 train.py/play.py source를 읽기
4. 원본의 import isaac_lab_tasks 바로 뒤에 import rok4_tasks 삽입
5. train.py에는 RoK4OnPolicyRunner import/생성을 삽입
6. teleop이면 device CLI, SE(2) 입력 생성, command update를 play.py에 삽입
7. estimator teleop이면 current observation의 추정 base XY 속도를 주황색 marker로 표시
8. keyboard teleop이면 R key의 one-shot environment reset request를 play loop에 삽입
9. 원본 script를 실행

이 방식의 장점은 Isaac Lab 본체를 수정하지 않는다는 점이다.

scripts/rsl_rl/train.py

RoK4 task와 RoK4OnPolicyRunner 를 등록한 뒤 Isaac Lab 원본 RSL-RL training script를 실행한다.

사용자는 Isaac Lab root에서 다음처럼 실행한다.

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 512 \
--max_iterations 5000 \
--headless
```

scripts/rsl_rl/play.py

RoK4 task를 등록한 뒤 Isaac Lab 원본 RSL-RL playback script를 실행한다.

사용자는 Isaac Lab root에서 다음처럼 실행한다.

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/play.py \
--task RoK4-Isaac-Velocity-Flat-Play-v0 \
--num_envs 16 \
--checkpoint /path/to/model.pt
```

이 local launcher는 Isaac Lab 원본 play loop에 `RoK4PushTestWindow.apply_pending_push()` 호출을 삽입한다. 따라서 UI callback은 simulation tensor를 직접 바꾸지 않고 push 요청만 queue하며, 실제 root velocity 변경은 다음 policy-step 경계에서 실행된다.

Velocity command monitor와 manual push test UI

`push_test_window.py` 의 `RoK4PushTestWindow` 는 Isaac Lab의 `ManagerBasedRLEnvWindow` 를 상속하고 기존 Isaac Lab 창에 `RoK4 Velocity Monitor` 와 `RoK4 Push Test frame`을 추가한다.

`RoK4 Velocity Monitor` 는 `command_manager.get_command("base_velocity")` 에서 Viewer가 선택한 environment의 최종 command를 읽어 command `vx`, `vy` [m/s], `wz` [rad/s], `|vxy|` [m/s]를 표시한다. Teleop에서는 gamepad/keyboard 입력을 학습 범위로 scale하고 부호를 변환한 뒤 command buffer에 실제로 기록한 값이다.

실제 로봇 속도는 world linear velocity를 tracking reward의 `track_lin_vel_xy_yaw_frame_exp` 와 동일하게 gravity-aligned yaw frame으로 회전하여 `vx`, `vy`, `vz` [m/s]와 `|vxy|` [m/s]를 표시한다. 실제 yaw 각속도는 `track_ang_vel_z_world_exp` 와 같은 world Z축 `wz_world` [rad/s]다. 따라서 command와 실제 추종 결과를 같은 화면에서 직접 비교할 수 있다. UI는 physics 500 Hz가 아니라 20 Hz로 갱신하여 불필요한 GPU-to-CPU synchronization을 줄인다. Play가 여러 환경을 사용할 때는 Viewer Settings > Environment Index 에 맞춰 표시 대상도 바뀐다.

RoK4 Push Test 는 별도의 키보드/gamepad mapping 없이 마우스로 다음 버튼을 제공한다.

- +X, -X: 로봇의 현재 전방/후방 방향으로 root 선속도를 변경
- +Y, -Y: 로봇의 현재 좌측/우측 방향으로 root 선속도를 변경
- Random XY: X/Y 속도 변화를 각각 독립적으로 [-magnitude, magnitude] 에서 표본화
- Delta velocity [m/s]: 0.05-1.0 m/s 범위의 변화량, 기본값 0.5 m/s

버튼이 만드는 값은 base-yaw frame의 $\Delta \mathbf{v}_b = [\Delta v_x, \Delta v_y, 0]$ 이다. 실제 적용 시점의 root quaternion에서 yaw만 취한 $R_{wb, yaw}$ 로

$$\Delta \mathbf{v}_w = R_{wb, yaw} \Delta \mathbf{v}_b$$

를 계산하고, 선택한 환경의 현재 `root_vel_w=[vx,vy,vz,wx,wy,wz]` 중 선속도에 더한 뒤 `write_root_velocity_to_sim()` 으로 기록한다. Roll/pitch를 제거하므로 로봇이 기울어도 외란은 수평이고, 로봇이 회전한 뒤에도 버튼의 전후좌우 의미는 유지된다. 이는 newton 단위의 일정 시간 외력을 적분하는 방식이 아니라, 충격 외란과 비슷한 순간 root 속도 변화다. velocity command 자체는 바꾸지 않는다.

Play가 여러 환경을 사용할 때는 Viewer Settings > Environment Index 로 선택한 환경 하나에만 적용한다. Teleop은 env가 하나이므로 항상 env 0이다. 학습 중 부모 `push_robot` event는 각 환경에 10-15초 간격으로 독립적인 X/Y 속도 외란을 자동 적용하지만, Play/Teleop에서는 자동 event를 끄고 이 수동 버튼만 사용한다. 버튼 UI는 GUI가 있는 local `play.py` 및 `play_teleop.py` wrapper에서만 동작하며 headless 학습에는 생성되지 않는다. Isaac Lab 원본 파일은 수정하지 않는다.

Isaac Lab panel의 x 를 누르면 window object가 삭제되는 것이 아니라 숨겨진다. Isaac Sim 상단 메뉴의 Window > IsaacLab 을 선택하면 같은 panel을 다시 표시하고 오른쪽 Property tab 위치로 재도킹한다. 실행 중인 구버전 코드에서 임시로 panel만 복구할 때는 Python Console에서 `omni.ui.Workspace.get_window("IsaacLab").visible = True` 를 실행할 수 있지만, 자동 재도킹은 RoK4 callback이 있는 새 실행부터 적용된다.

Contact force debug visualization

`contact_force_visualizer.py` 의 `RoK4ContactForceVisualizer` 는 Isaac Lab의 기존 `ContactSensor` 를 상속한다. 보상과 termination은 기존 `contact_forces` sensor data를 그대로 사용하고, debug visualization을 켜둘 때만 환경 0의 좌우 발 데이터를 읽어 다음 요소를 표시한다.

- 원발: 파란색 world-frame 전체 지면반력 화살표
- 오른발: 초록색 world-frame 전체 지면반력 화살표
- RoK4 Contact Forces 창: 좌우 발의 $|F|$ 를 newton 단위 숫자로 표시
- 같은 창의 live plot: 좌우 $|F|$ 최근 3.0초, 301개 sample을 두 개의 선으로 표시
- live plot X축: 0.1 s 간격 세로 grid와 0.5 s 간격 숫자로 누적 physics time [s]를 표시

Isaac Sim UI의 Scene Debug Visualization 에서 Contact Forces 를 체크하면 화살표, 숫자, live plot이 함께 켜지고, 체크를 해제하면 함께 숨겨진다. Flat task의 지면 collision prim을 filter로 지정하고 PhysX가 별도로 제공하는 world-frame 법선 접촉력과 접선 접촉력을 발별로 더한다.

창 기본 크기는 600 x 650 이며 제목, 좌우 force 숫자, X/Y축 label을 읽기 쉬운 크기로 표시한다. 행 간격과 내부 여백을 줄이고 좌우 force 숫자 및 범례를 가까이 묶어 viewport를 덜 가린다. Y축은 0~4000 N 고정 범위와 1000 N 간격 눈금을 사용한다. Isaac Lab LiveLinePlot 은 불필요한 filter, integration, derivative, autoscale, editable-limit UI를 자동으로 붙이므로 사용하지 않는다. RoK4 전용 graph는 두 개의 force trace와 다섯 개의 고정 Y-grid를 동일한 omni.ui.Plot 좌표계로 겹쳐 그린다. 따라서 force 선과 0/1000/2000/3000/4000 N 눈금이 정확히 일치한다. X축 세로 grid도 같은 omni.ui.Plot 좌표계에서 histogram line으로 그린다. 최근 3.0초를 0.1 s 간격 minor tick 30개로 나누고, 매 0.5 s major tick에는 더 밝은 선과 시간 숫자를 표시한다. X축 시간은 ContactSensor의 누적 physics time으로 계산한다. 환경 reset으로 센서 timestamp가 0으로 돌아가더라도 이전 elapsed time에 새 timestamp를 이어 붙이므로 1초 timeline 구간이나 episode reset에서 0으로 반복되지 않는다.

$$\mathbf{F}_{GRF}^w = \mathbf{F}_{normal}^w + \mathbf{F}_{tangential}^w = [F_x^w, F_y^w, F_z^w]$$

발마다 이 합력 벡터 하나를 표시한다. 화살표 방향은 $\mathbf{F}_{GRF} / |\mathbf{F}_{GRF}|$ 이고, 화살표 길이와 숫자는 $|\mathbf{F}_{GRF}| = \sqrt{F_x^2 + F_y^2 + F_z^2}$ 에 비례한다. X/Y/Z 축마다 별도 화살표를 만드는 구조가 아니다. 기본 arrow_x.usd 의 local x 범위가 [-0.25, 0.75] 이므로 화살표 원점을 합력 방향으로 보정하여 꼬리가 지면 아래로 들어가지 않고 발바닥 바로 위에서 시작하도록 한다. 가독성을 위해 화살표 길이에 display-only 상한을 적용한다.

이 값은 발과 지면 사이의 전체 접촉 합력이며, 발목에 설치한 6축 F/T sensor의 $[F_x, F_y, F_z, M_x, M_y, M_z]$ 출력은 아니다. 4096-env 학습에서 불필요한 GPU-to-CPU/UI 비용이 발생하지 않도록 debug view 기본값은 off이며, 켜를 때도 환경 0만 시각화한다. 구현과 설정은 모두 rok4_lab 안에 있고 Isaac Lab 원본은 수정하지 않는다. Live plot은 render update마다 최신 sensor sample을 추가하므로 60 Hz rendering에서는 최근 약 5초를 보여주지만, 500 Hz physics substep impact peak를 보존하는 graph는 아니다.

scripts/rsl_rl/play_teleop.py

기존 Play task는 변경하지 않고, Teleop task에서만 학습된 policy의 velocity command를 수동 입력으로 교체한다. Se2Gamepad 는 Omniverse gamepad interface를 직접 사용하므로 ROS 2 joy_node, Isaac Sim ROS 2 Bridge, /joy subscriber 또는 별도 IPC가 필요하지 않다.

Gamepad 실행:

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/play_teleop.py \
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \
--teleop_device gamepad \
--teleop_dead_zone 0.05 \
--checkpoint /path/to/model.pt \
--real-time
```

Gamepad의 left stick 위/아래는 x 선속도를 명령한다. Left stick 오른쪽은 음의 y 선속도, 왼쪽은 양의 y 선속도이며, right stick 오른쪽은 음의 yaw, 왼쪽은 양의 yaw를 명령하므로 스틱 방향과 로봇의 이동/회전 방향이 일치한다. 정규화된 입력은 [-1, 1] 로 clip한 뒤 학습 범위 $v_x=(-0.3, 0.85)$ m/s, $v_y=(-0.3, 0.3)$ m/s, $w_z=(-0.6, 0.6)$ rad/s 로 scale한다.

Keyboard 실행:

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/play_teleop.py \
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \
--teleop_device keyboard \
--teleop_keyboard_step 0.05 \
--checkpoint /path/to/model.pt \
--real-time
```

Keyboard는 Up/Down으로 전진/후진, Left/Right로 좌/우 횡이동, Z/X 로 양/음 yaw를 명령한다. L 은 keyboard command만 [0,0,0] 으로 초기화한다. R 은 one-shot reset request를 기록하고, 다음 policy loop가 device command, simulation environment, policy state를 함께 초기화한다. Environment tensor는 기존 play loop의 env.step() 에서 inference tensor로 생성되므로 수동 env.reset() 도 반드시 torch.inference_mode() 안에서 실행한다. 그렇지 않으면 inference tensor

inplace-update 예외로 play가 종료된다. 키 입력 전에 Isaac Sim viewport를 클릭해 keyboard focus를 주어야 한다. 각 key press는 기본적으로 선속도 축을 0.05 m/s, yaw를 0.05 rad/s 씩 누적한다. 키를 놓아도 command는 유지되고 반대 방향 key를 누르면 같은 크기만큼 감소한다. 누적값은 각 축의 학습 command 범위에서 clamp되며, --teleop_keyboard_step 으로 공통 수치 increment를 변경할 수 있다.

Teleop command는 100 Hz play loop 시작 시 command buffer에 기록된다. 다만 그 시점의 policy observation은 직전 environment step에서 이미 생성되어 있으므로 현재 action에는 직전 command observation이 사용된다. 새 command는 이어지는 step 후 observation을 통해 다음 policy loop에서 보이며, 수동 입력부터 policy 반영까지 최대 한 policy period인 약 10 ms 가 걸린다.

Teleop에서도 같은 command/actual 숫자 panel과 RoK4 Push Test frame이 표시된다. 조이스틱/키보드로 이동 command를 유지한 상태에서 실제 scale된 command와 로봇의 측정 속도를 비교하고, 마우스로 방향 버튼을 눌러 command tracking과 외란 복원을 동시에 확인할 수 있다.

Concurrent estimator checkpoint를 Teleop으로 실행하면 로봇 위의 평면 선속도 화살표가 세 개가 된다.

- 녹색: body-frame command $[v_x, v_y]$
- 파란색: simulator ground truth $root_lin_vel_b[0:2]$
- 주황색: Actor가 사용한 estimator $[\hat{v}_x, \hat{v}_y]$

세 값은 같은 body frame이며 robot orientation으로 world에 회전하고, 같은 $3|v_{xy}|$ 길이 배율을 사용한다. 주황색 marker는 기존 두 marker보다 0.08 m 높여 겹침을 구분한다. $\hat{v}_z$ 는 command에 대응하는 값이 없으므로 3D 화살표에 섞지 않고 ONNX diagnostic output과 estimator RMSE에서 확인한다. 시각화는 current policy observation으로 estimator를 한 번 더 결정적으로 실행하는 진단 경로이며 학습, action, reward 및 ONNX 출력을 변경하지 않는다. 구현은 `scripts/rsl_rl/_velocity_estimate_visualizer.py`에 있다.

수정된 기존 파일

source/rok4_tasks/rok4_tasks/__init__.py

기존에는 패키지 docstring만 있었다. 이제 RoK4 task registry가 import되도록 다음 역할을 한다.

```
from .manager_based.locomotion.velocity.config import rok4
```

이 import가 실행되어야 RoK4-Isaac-Velocity-Flat-v0 task가 gym registry에 등록된다.

README.md

새 Flat RL Task 섹션을 추가했다. task 이름, observation/action 구조, smoke test, normal training, play 명령어, DR 관리 파일, self-collision 설정을 문서화했다. Teleop task, gamepad/keyboard 입력, command scale과 한 policy-step 입력 지연도 함께 문서화했다. Contact Forces debug toggle로 환경 0의 좌우 발 접촉력 화살표와 실시간 newton 값을 확인하는 방법도 문서화했다. Play/Teleop의 command/actual 속도 숫자 panel과 RoK4 Push Test 버튼, base-yaw-frame Delta v 의미, 선택 environment 및 queue 적용 흐름도 문서화했다. 이후 수동 push를 base-yaw frame으로 변경하고 Window > IsaacLab panel 복구 및 오른쪽 Property tab 재도킹 경로를 추가했다.

CHANGELOG.md

Unreleased 항목에 RoK4 flat RSL-RL task 추가 내용을 기록했다.

학습 실행 흐름

학습 명령을 실행하면 전체 흐름은 다음과 같다.

```
./isaacclab.sh
-> ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py
-> _run_isaacclab_rsl.py
-> sys.path에 rok4_tasks 추가
-> Isaac Lab 원본 train.py 실행
-> import isaacclab_tasks
-> import rok4_tasks
-> RoK4 task gym 등록
-> hydra_task_config가 env_cfg / agent_cfg 로드
-> gym.make("RoK4-Isaac-Velocity-Flat-v0")
-> ManagerBasedRLEnv 생성
-> RslRLVecEnvWrapper
-> RoK4OnPolicyRunner
-> RoK4PPO 학습
-> Loss/kl, Loss/kl_max 기록
```

스모크 테스트 결과

다음 smoke test를 수행했다.

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 2 \
--max_iterations 1 \
--headless
```

확인된 결과는 다음과 같다.

항목	결과
환경 생성	성공
action shape	13
observation shape	이전 smoke 기준 policy=240, privileged=3
actor input	5-step history 기반 240차원
critic input	이전 smoke 기준 actor history와 현재 base_lin_vel_b 를 합친 243차원
actor output	13차원 normalized actuator action
PPO 1 iteration	성공
생성 checkpoint	\${ISAACLAB_ROOT}/logs/rsl_rl/rok4_flat/2026-07-07_13-44-26/model_0.pt

Directional gait 20k 검증 기준

현재 directional-gait 기준 run은 다음과 같다.

```
run      : 2026-08-03_14-58-46_touchdown_air_symmetric_x_fastforward_fresh20k
checkpoint : model_19999.pt
branch   : yunho/directional-gait-rework
```

Teleop에서 전진, 후진, 좌우 횡이동, 양방향 yaw 회전을 확인했다. 아래 값은 각 checkpoint step에서 기록된 TensorBoard scalar이며, reward 설정이 다른 run과 Train/mean_reward 만 직접 비교해서는 안 된다.

Directional gait checkpoint 비교

checkpoint	mean reward	XY velocity error	yaw velocity error	feet slide	policy noise std
model_9999.pt	37.82	0.114	0.149	-0.00564	0.0709
model_14999.pt	38.34	0.091	0.138	-0.00417	0.0669
model_19999.pt	38.35	0.093	0.139	-0.00343	0.0697

Tracking은 약 15k에서 사실상 plateau에 도달했고, 15k에서 20k 사이에는 sliding, jump, orientation 같은 contact-related term이 조금 더 정돈되었다. 따라서 현재 기준 checkpoint는 model_19999.pt 로 보존하되, 새 실험에서도 model_9999.pt, model_14999.pt, model_19999.pt 를 모두 비교한다.

yunho/privileged-observation branch는 위 directional baseline에서 critic을 250D로 확장한 Sim2Real baseline이다. 현재 yunho/concurrent-state-estimator branch는 여기서 다시 분기해 command-free 225D estimator를 추가한다. Actor의 외부 240차원 입력과 13차원 actuator action contract, Critic 250D는 유지하며 이전 checkpoint를 resume하지 않고 fresh 학습한다.

현재 설계 의도

현재 task는 최종 rough terrain 정책이 아니다. 첫 번째 목표는 원인 분리가 쉬운 flat walking baseline이다.

현재 단계에서 의도적으로 넣지 않은 항목은 다음과 같다.

```
rough terrain
terrain curriculum
height scanner
camera observation
teacher-student distillation
terrain height scanner / rough-terrain privileged height state
```

추후 권장 순서는 다음과 같다.

1. Flat smoke test 확인
2. Flat 학습으로 서기/걸기 안정화
3. reward, action scale, termination 조정
4. RoK4 Rough task 추가
5. weak rough terrain curriculum
6. concurrent base-velocity estimator 검증
7. estimator 기반 stable-standing/recovery gate ablation
8. 필요 시 teacher-student 구조 추가

주요 실행 명령어

Smoke test:

```
export ROK4_LAB_ROOT="${HOME}/rok4_lab"
export ISAACLAB_ROOT="${HOME}/IsaacLab"
cd "${ISAACLAB_ROOT}"
conda activate env_isaaclab

./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 2 \
--max_iterations 1 \
--headless
```

Normal training:

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 4096 \
--max_iterations 5000 \
--headless \
--run_name symmetry_aug_fresh
```

Training with periodic video:

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 4096 \
--max_iterations 5000 \
--headless \
--video \
--video_length 500 \
--video_interval 10000 \
--run_name symmetry_aug_fresh_video
```

video_length 와 video_interval 은 PPO iteration이 아니라 environment/policy step 기준이다. 현재 100 Hz policy에서 위 설정은 5초 영상을 100초의 simulation time마다 기록하며 run folder의 videos/train/ 에 저장한다. Headless 녹화는 config에 설정된 고정 camera를 사용하므로 학습 중 대화식 zoom/회전은 할 수 없고, rendering으로 인해 GPU·학습시간·저장공간 overhead가 추가된다.

Play:

```
./isaacclab.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/play.py \
--task RoK4-Isaac-Velocity-Flat-Play-v0 \
--num_envs 16 \
--checkpoint /path/to/model.pt
```

실행 후 RoK4 Push Test frame에서 Delta velocity [m/s] 를 정하고 방향 버튼을 누른다. 여러 env를 띄운 경우 Viewer Settings > Environment Index 로 push 대상 env를 먼저 선택한다. 방향은 현재 base-yaw frame 기준이므로 로봇이 회전한 뒤에도 +x 는 로봇 전방이다. Panel을 닫았으면 Isaac Sim 상단의 Window > IsaacLab 으로 복구한다.

Teleop with gamepad:

```
./isaacsim.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/play_teleop.py \  
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \  
--teleop_device gamepad \  
--teleop_dead_zone 0.05 \  
--checkpoint /path/to/model.pt \  
--real-time
```

Teleop with keyboard:

```
./isaacsim.sh -p ${ROK4_LAB_ROOT}/scripts/rsl_rl/play_teleop.py \  
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \  
--teleop_device keyboard \  
--teleop_keyboard_step 0.05 \  
--checkpoint /path/to/model.pt \  
--real-time
```